

Customer Churn Prediction

A Machine Learning Project — Explained from Beginner to Implementation

Interview preparation & study guide

Python · pandas · scikit-learn · EDA · Feature Engineering · Model Evaluation

This document explains the project end to end — in plain language first, then at implementation depth — so you can confidently discuss every decision in an interview. All results come from actually running the code.

1. The 30-Second Pitch

If a recruiter asks “tell me about this project,” this is your answer:

“I built an end-to-end machine-learning system that predicts which telecom customers are about to cancel their service, so the company can reach out and keep them. I started with 7,000 real customer records, cleaned them, explored the data to find what drives churn, engineered new features, and trained several classification models inside a leakage-safe scikit-learn pipeline. I compared them with cross-validation and chose logistic regression because it caught about 80% of churners — the metric that matters most here — and because it's interpretable, so I can explain why a customer is flagged. The final model scores 0.85 ROC-AUC on unseen data.”

Everything in that paragraph is defended, in detail, in the rest of this document.

2. The Problem & Why It Matters

What is customer churn?

Churn means a customer leaving — cancelling their subscription. For a subscription business (telecom, streaming, SaaS), churn is one of the most important numbers there is. It is far cheaper to keep an existing customer than to acquire a new one, so if you can predict *who* is likely to leave *before* they do, you can offer them a discount, better plan, or support call and save the relationship.

What does the model actually do?

It takes a customer's details (how long they've been a customer, their contract type, monthly charges, which services they use, etc.) and outputs a **probability of churn** between 0 and 1. The business can then focus retention effort on the highest-risk customers.

This is a supervised classification problem

- **Supervised:** we have historical data where we already know who churned (labelled data).
- **Classification:** the output is a category — “will churn” or “won't churn” — not a number.
- **Binary:** exactly two classes (Yes / No).

3. The Dataset

The **IBM Telco Customer Churn** dataset — a well-known public dataset of **7,043 customers** and **21 columns**. Each row is one customer.

The columns fall into three groups

- **Demographics** — gender, senior citizen, partner, dependents.
- **Account information** — tenure (months as a customer), contract type, payment method, monthly charges, total charges.

- **Services subscribed** — phone, multiple lines, internet type, online security, backup, device protection, tech support, streaming TV/movies.

The **target** column is Churn (Yes/No). Critically, only about **27% of customers churned** — the classes are **imbalanced**, which shapes every later decision about metrics.

4. The End-to-End Workflow

The project follows the standard machine-learning pipeline. Each stage feeds the next, and everything that *learns* from the data is fit on the training portion only.

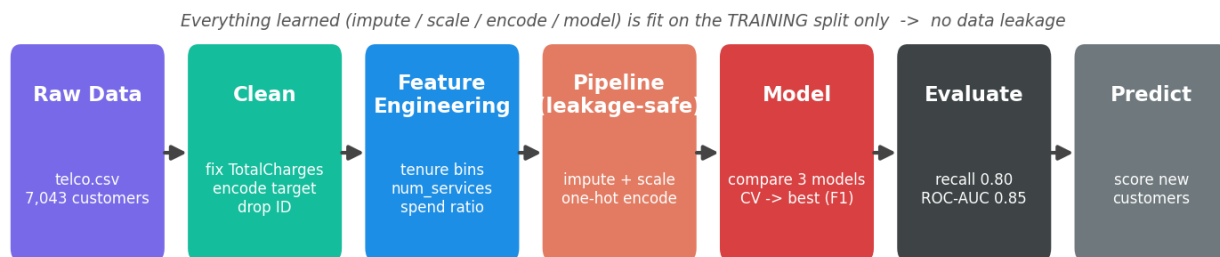


Figure 1 — the end-to-end pipeline, from raw CSV to live predictions.

5. Stage by Stage

5.1 Data Cleaning

Real data is messy. This dataset had two classic problems, both of which an interviewer loves to hear you spotted:

- **TotalCharges was stored as text** and contained 11 blank strings. Those 11 rows are all brand-new customers (tenure = 0) who haven't been billed yet, so the correct fill is 0 — a *reasoned* value, not a random guess.
- **The target was text** (“Yes”/“No”), so I encoded it to 1/0.
- **customerID is an identifier** with no predictive meaning, so I dropped it (leaving it in would let the model “memorise” customers).

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df.loc[df["tenure"] == 0, "TotalCharges"] = 0.0
df["Churn"] = (df["Churn"] == "Yes").astype(int)
```

5.2 Exploratory Data Analysis (EDA)

Before modelling, I explored the data to understand it and to spot problems. The single most important finding was the class imbalance.

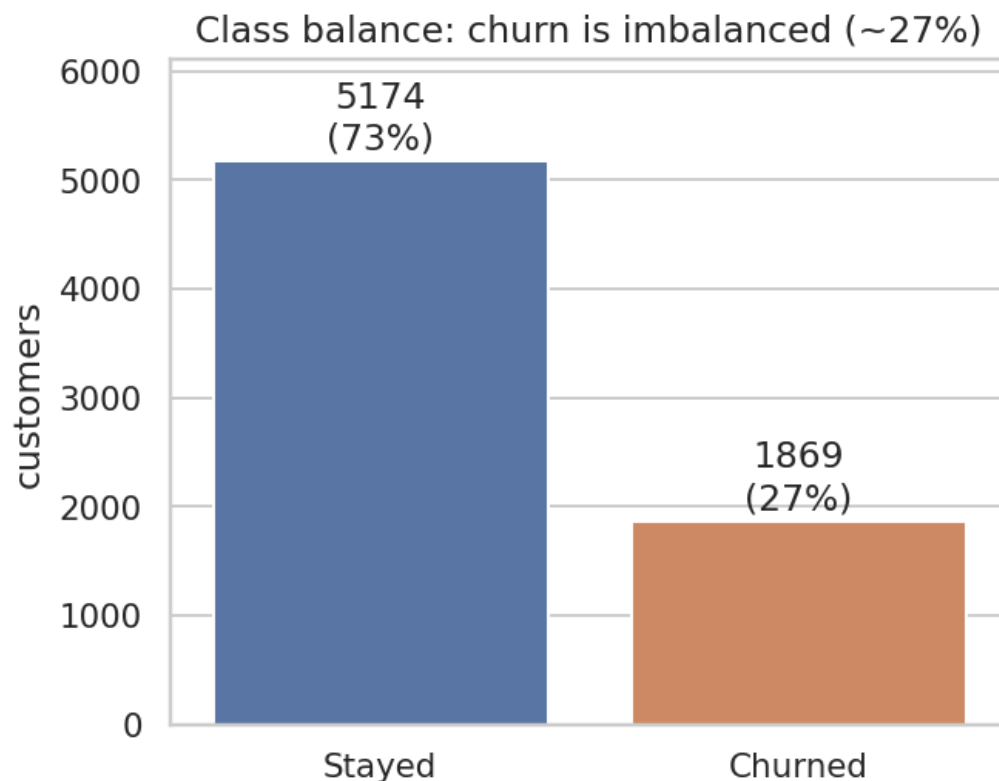


Figure 2 — only ~27% of customers churned. This is why accuracy is a misleading metric.

The strongest driver of churn is **contract type**: month-to-month customers churn far more than those on longer contracts, who are locked in and more committed.

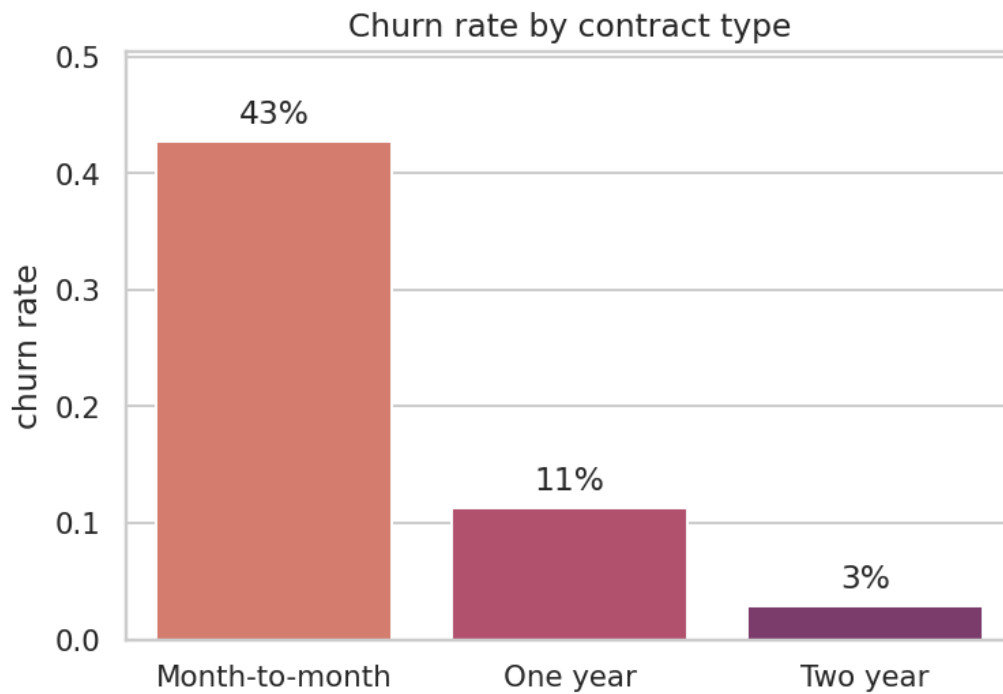


Figure 3 — month-to-month customers churn at ~43%; two-year customers at ~3%.

Tenure matters too: churn is concentrated in the first few months. New customers are the flight risk.

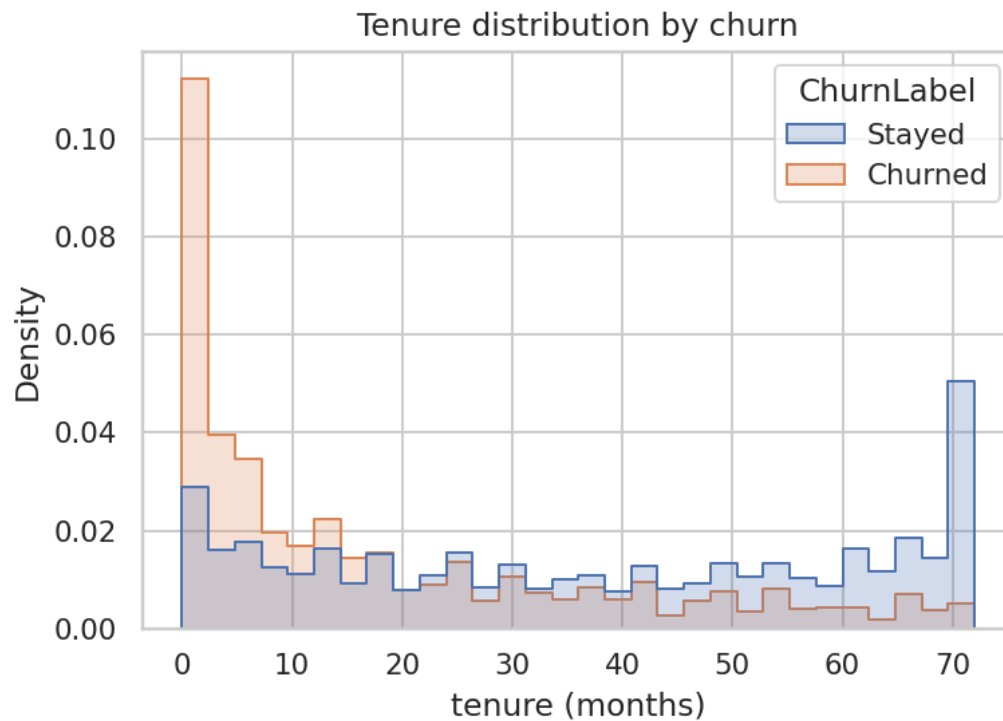


Figure 4 — churned customers (orange) cluster at low tenure; long-tenure customers rarely leave.

A **correlation heatmap** checks which numeric features move together. Note that tenure and total charges are correlated (longer customers have paid more) — a redundancy to be aware of.

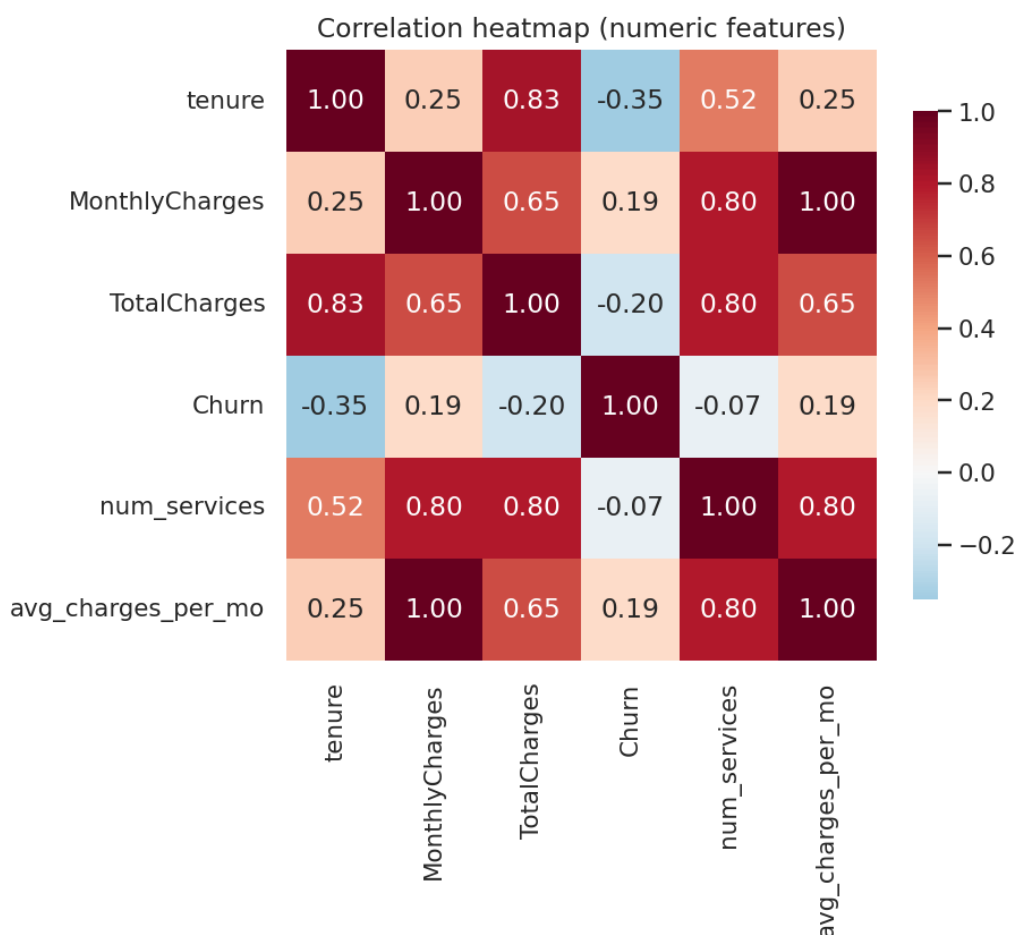


Figure 5 — correlations among numeric features (including the engineered ones).

5.3 Feature Engineering

Feature engineering — creating new columns that make patterns easier for the model to learn — is often the highest-leverage step in ML. I added four, each with a clear rationale:

- **tenure_group** — binning tenure into loyalty bands (0–1yr, 1–2yr, ...). Binning can expose non-linear relationships a raw number misses.
- **num_services** — an *aggregation*: how many paid services the customer has. More services usually means a “stickier,” harder-to-leave customer.
- **avg_charges_per_mo** — a *ratio* ($\text{TotalCharges} \div \text{tenure}$): the customer's typical lifetime monthly spend.
- **is_new_customer** — a flag for $\text{tenure} \leq 6$ months, since churn is heavily concentrated there.

Proof they were worth it: two of these engineered features (num_services and is_new_customer) show up among the model's most important features (Figure 8). That's the “my features earned their place” story interviewers want to hear.

5.4 Preprocessing — the leakage-safe Pipeline

This is the part that separates a beginner from someone who understands ML properly. All preprocessing that *learns* anything from the data — filling missing values, scaling numbers, one-hot encoding categories — is wrapped in a scikit-learn Pipeline with a ColumnTransformer, and fitted on the **training split only**.

Why this matters: if you compute a scaler's mean using the whole dataset, information from the test set “leaks” into training and your scores look great in your notebook but collapse in production. Fitting inside a pipeline makes leakage impossible by construction.

```
num_pipe = Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())])
cat_pipe = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                     ("onehot", OneHotEncoder(handle_unknown="ignore"))])
pre = ColumnTransformer([("num", num_pipe, numeric_cols),
                        ("cat", cat_pipe, categorical_cols)])
model = Pipeline([("pre", pre), ("clf", LogisticRegression(...))])
```

5.5 Model Selection

I compared three standard classifiers, each cross-validated (5-fold, stratified) on the training set. I looked at ROC-AUC, F1, and recall — **not** accuracy, because of the imbalance.

Model	ROC-AUC	F1	Recall
Logistic Regression	0.847	0.628	0.794
Random Forest	0.829	0.544	0.470
Gradient Boosting	0.847	0.587	0.529

Decision: Logistic Regression won on F1 and had by far the best recall (0.79). For churn, recall is the priority — a missed churner is a lost customer, which costs more than a false alarm. It is also the most interpretable model, so I can explain which factors push a customer toward leaving. (Gradient Boosting tied on ROC-AUC but had much lower recall.)

5.6 Evaluation on Unseen Data

The real test is performance on the held-out 20% the model never saw during training.

Metric	Score	Interpretation
ROC-AUC	0.845	ranks churner above non-churner 85% of the time
Recall	0.80	catches 80% of customers who actually churn
Precision	0.50	half of flagged customers really churn
F1	0.62	balance of precision and recall
Accuracy	0.74	reported, but NOT the target (data is imbalanced)

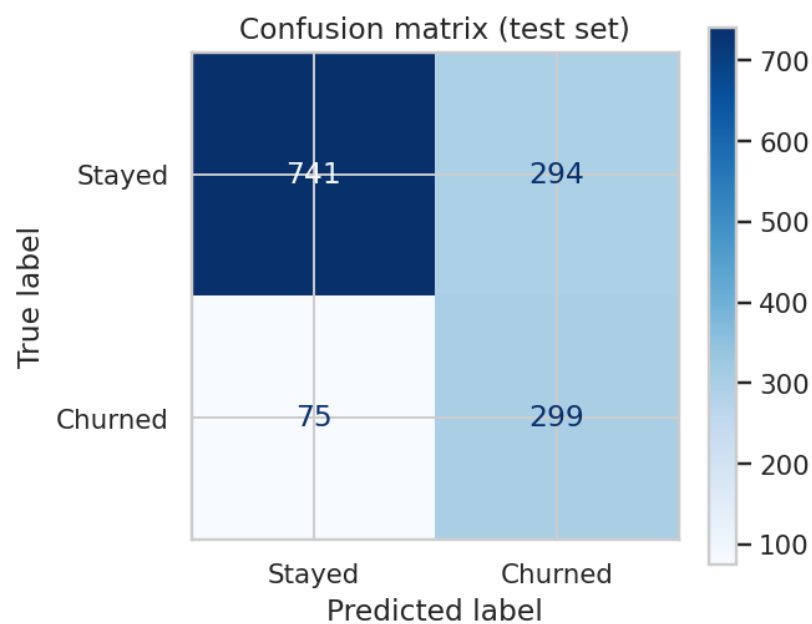


Figure 6 — of 374 real churners the model catches 299 and misses only 75 (80% recall).

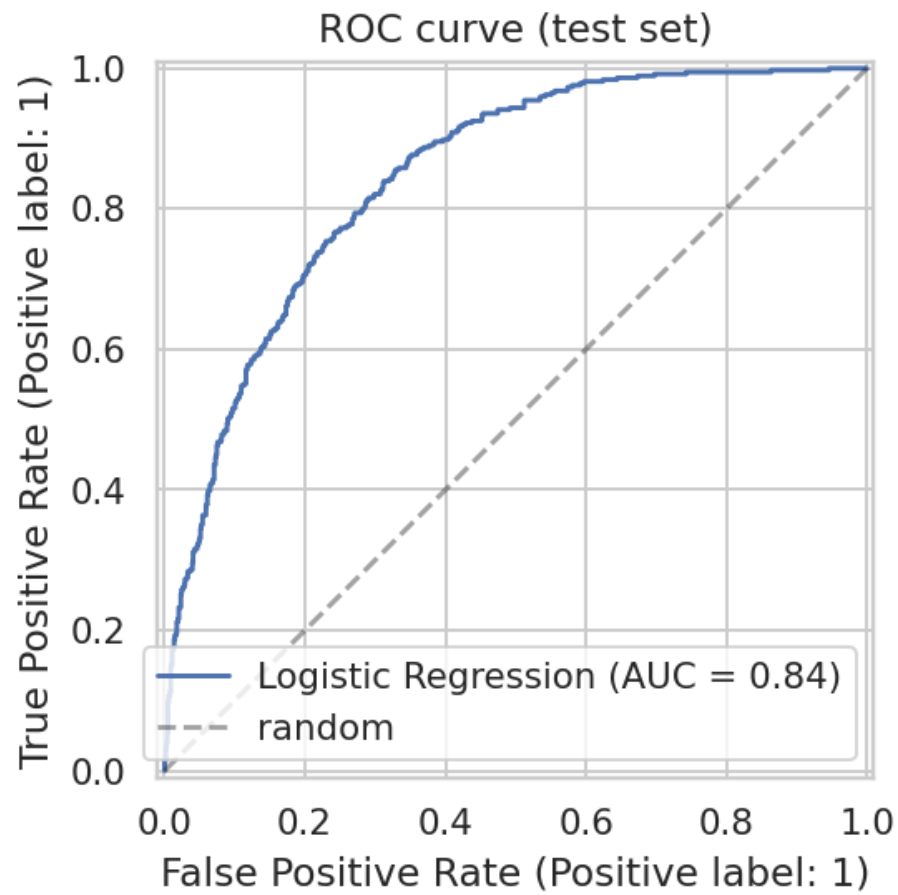


Figure 7 — ROC curve. Area under it (0.85) summarises ranking ability across all thresholds.

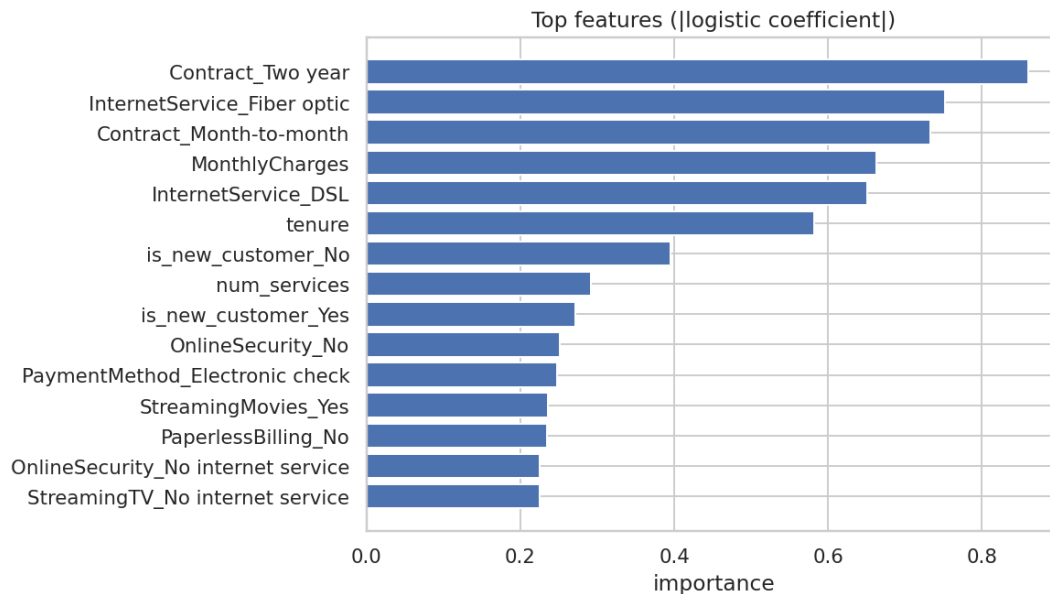


Figure 8 — what drives the prediction. Contract, internet type, charges, tenure dominate; engineered features num_services and is_new_customer earned their place.

6. Key Concepts — Explained Simply

These are the ideas an interviewer will probe. Know them cold.

Class imbalance. When one class is much rarer than the other (here 27% churn). A lazy model that always predicts “no churn” is 73% accurate but catches zero churners — useless. This is why we don't use accuracy.

Precision. Of the customers the model *flagged* as churners, what fraction actually churned. High precision = few false alarms.

Recall. Of the customers who *actually* churned, what fraction did the model catch. High recall = few missed churners. For churn, this is usually the priority.

F1 score. The harmonic mean of precision and recall — a single number that balances both. Good default for imbalanced classification.

ROC-AUC. Measures how well the model *ranks* customers by risk, independent of any threshold. 0.5 = random, 1.0 = perfect. 0.85 here is solid.

Data leakage. When information from the test set (or the future) sneaks into training, inflating scores that then collapse in production. Prevented by fitting all learned steps on the training split only — which a Pipeline enforces automatically.

One-hot encoding. Turning a categorical column (e.g. Contract) into 0/1 indicator columns, one per category. Used for unordered categories so the model doesn't invent a false ranking.

Feature scaling. Putting numeric features on a comparable range (mean 0, std 1). Needed by distance/gradient models like logistic regression; tree models don't need it.

Cross-validation. Splitting the training data into k folds and rotating which fold is validation, to get a more reliable performance estimate than a single split. “Stratified” keeps the churn ratio the same in every fold.

Class weights. Telling the model to pay more attention to the rare class during training (class_weight=“balanced”), so it doesn't ignore churners. My chosen alternative to resampling.

7. Common Interviewer Questions & Answers

Practice saying these out loud. The answers are honest and match what the code actually does.

Q: Why did you choose accuracy... wait, you didn't. Why not?

A: Because the data is imbalanced (73% stay / 27% churn). A model that always predicts “stay” would be 73% accurate but catch no churners. I optimised for F1 and recall, which measure how well the model actually finds churners.

Q: Why is recall the most important metric here?

A: Because the cost of a false negative (missing a customer who churns) is high — you lose that customer's revenue. A false positive (offering retention to someone who would have stayed) is comparatively cheap. So I'd rather catch more churners even at the cost of some false alarms.

Q: Why did you pick Logistic Regression over Random Forest or Gradient Boosting?

A: On cross-validation it had the best F1 and clearly the best recall (0.79 vs 0.53 for boosting). It's also interpretable — the coefficients tell me which factors drive churn — which is valuable for a business that wants to act on the predictions, not just receive a score.

Q: How did you prevent data leakage?

A: I put every learned transformation — imputation, scaling, one-hot encoding — inside a scikit-learn Pipeline and fit it on the training split only. The test set is transformed using parameters learned from training, never its own. Row-wise fixes that don't learn anything (type conversion, dropping the ID) are safe to do beforehand.

Q: How did you handle the missing values in TotalCharges?

A: The 11 missing values were all customers with tenure 0 — brand-new, not yet billed — so I filled them with 0, which is the correct value, not a statistical guess. I kept a median-fill as a safety net for any other case. I also converted the column from text to numeric first, since it was stored as strings.

Q: What features did you engineer, and did they help?

A: Four: a tenure band (binning), a count of services (aggregation), average monthly spend (a ratio), and a new-customer flag. Two of them — number of services and the new-customer flag — ranked among the model's top features, so yes, they added value.

Q: Which models need feature scaling and which don't?

A: Distance- and gradient-based models — logistic regression, SVM, KNN, neural nets — need scaling. Tree-based models (decision trees, random forests, gradient boosting) don't, because they split on thresholds and are invariant to feature scale. I scaled anyway inside the pipeline; it helps logistic regression and is harmless for the trees.

Q: What is one-hot encoding and why not just use integers?

A: One-hot creates a 0/1 column per category. If I mapped an unordered category like payment method to integers (0,1,2,3), the model would wrongly assume an order and that category 3 is “more”

than category 1. One-hot avoids that false ordering.

Q: How did you deal with class imbalance?

A: I used class weights (`class_weight="balanced"`), which makes the model penalise mistakes on the rare churn class more heavily during training. It's simple, keeps the pipeline clean, and avoids the risks of naive oversampling. Resampling (SMOTE) would be a valid alternative to try.

Q: What does ROC-AUC of 0.85 actually mean?

A: If I pick a random churner and a random non-churner, the model gives the churner a higher risk score about 85% of the time. It measures ranking quality across all thresholds, independent of where I set the 0.5 cut-off.

Q: How would you set the decision threshold in practice?

A: 0.5 is just a default. In practice I'd work with the business: if retention offers are cheap and losing a customer is expensive, lower the threshold to catch more churners (higher recall). I'd plot the precision-recall trade-off and pick the point that matches the cost of a false positive versus a false negative.

Q: How do you know the model isn't overfitting?

A: The cross-validation scores on training folds (ROC-AUC 0.847) and the held-out test score (0.845) are almost identical, which indicates the model generalises rather than memorising. The low variance across CV folds (std ~0.01) supports that too.

Q: How would you deploy this model?

A: The whole pipeline is one object saved with joblib. In production I'd load it behind an API or a batch job, pass in customer records, and get probabilities back — the exact same preprocessing is applied automatically, so training and serving can't drift apart. Monitoring would watch for data drift and performance decay over time.

Q: What would you do to improve it?

A: Tune the decision threshold to the business cost; run a hyperparameter search inside the pipeline; try gradient-boosting libraries like XGBoost/LightGBM; add SHAP values for per-customer explanations; and, if there were high-cardinality categories, use out-of-fold target encoding.

Q: Why did you use cross-validation instead of a single train/test split for model selection?

A: A single split can be lucky or unlucky. 5-fold cross-validation averages performance over five different validation sets, giving a more reliable comparison between models. I kept a separate held-out test set that was only used once, at the very end, for the final honest estimate.

Q: Is this project your own work if it was scaffolded with help?

A: The honest answer: I understand every stage — I can explain the cleaning choices, why each feature exists, why logistic regression, why recall, and how the pipeline prevents leakage. I've run it, read the code, and can modify it. That understanding is what makes it mine.

Built as a learning project on the public IBM Telco Customer Churn dataset. All metrics are reproducible by running `python main.py` with the fixed random seed.